Name: P.Sai Venkatesh

H.No: 2303A53039 - B46

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance.
- Apply **modern Python best practices** while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
  - Prompt AI to identify duplicate logic and replace it with functions or classes.
  - Ensure the refactored code maintains the same output.
  - Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
  - Refactored code with a reusable function and no duplication.

Prompt Used:

Refactor the following Python code by removing duplicate logic.
Convert repeated calculations into a reusable function.
Ensure the output remains exactly the same and add proper docstrings.

Code:

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Code:

```
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, width in rectangles:
    area, perimeter = calculate_rectangle_properties(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
```

Output:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

- Task: Use AI to analyze a Python script with nested loops and complex conditionals.
- Instructions:
  - Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).
  - Implement changes while keeping logic intact.
  - Compare execution time before and after refactoring.
- Sample Legacy Code:

```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

- Expected Output:
  - Optimized code using set lookups with performance comparison table.

Prompt Used:

Analyze the following Python code and optimize it by reducing time complexity.
Replace nested loops with efficient structures like sets or dictionaries.
Keep the logic unchanged and also provide a comparison of time complexity before and after optimization.

Code:

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
```

```
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list”)
```

Code:

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
name_set = set(names)  # Convert list to set for O(1) average time complexity
for s in search_names:
    if s in name_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
# Time Complexity Comparison:
# Before Optimization: O(n * m) where n is the length of 'names' and m is the length of
'search_names' due to nested loops.
# After Optimization: O(n + m) where n is the time to create the set from 'names' and m is the
time to check each element in 'search_names' against the set.
```

Output:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Task Description #3 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:

    - Identify repeated or related logic and extract it into reusable functions.
    - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- Expected Output:
    - Code with a function calculate_total(price) that can be reused for multiple price inputs.

Prompt Used:

Refactor the following Python code by extracting repeated logic into a reusable function named calculate_total(price).
Ensure the function includes a docstring and the output remains unchanged.

Code:

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Code:

```
def calculate_total(price):
    """
    Calculate the total price including tax.
```

```
    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
prices = [250, 500]
for price in prices:
    total = calculate_total(price)
    print("Total Price:", total)
```

Output

```
Total Price: 295.0
Total Price: 590.0
```

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Use AI to identify and replace all hardcoded "magic numbers" in the code with named constants.
- Instructions:
  - Create constants at the top of the file.
  - Replace all hardcoded occurrences in calculations with these constants.
  - Ensure the code remains functional and is easier to maintain.
- Sample Legacy Code:

```
# Legacy script with hardcoded values
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

- Expected Output:
  - Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.

Prompt Used:

Refactor the following Python code by replacing all hardcoded values with named constants. Define constants at the top of the file and ensure the program remains functional.

Code:

```
print("Area of Circle:", 3.14159 * (7 ** 2))
```

print("Circumference of Circle:", 2 * 3.14159 * 7)

Code:

```
import math
PI = math.pi
RADIUS = 7
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

Output:

```
Area of Circle: 153.93804002589985
Circumference of Circle: 43.982297150257104
```

---

ask Description #5 (Refactoring – Improving Variable Naming and Readability)

- Task: Use AI to improve readability by renaming unclear variables and adding inline comments.
- Instructions:
  - Replace vague names with meaningful ones.
  - Add comments where logic is not obvious.
  - Keep functionality exactly the same.
- Sample Legacy Code:

```
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)
```

- Expected Output:
  - Code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.

Prompt Used:

Improve the readability of the following Python code by renaming variables with meaningful names and adding inline comments.
Do not change the functionality.

Code:

```
a = 10
```

```
b = 20
c = a * b / 2
print(c)
```

Code:

```
base_length = 10  # Length of the base of the triangle
height_length = 20  # Length of the height of the triangle
area_of_triangle = (base_length * height_length) / 2  # Calculate the area of the triangle using the formula (base * height) / 2
print(area_of_triangle)
```

Output:

```
100.0
```

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

- **Task: Use AI to refactor a Python script that contains repeated if–else logic for grading students.**
- Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

o Ensure grading logic remains unchanged.

- Code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
```

T **print("Grade B")**

```
else:
    print("Grade C")

marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
```

```
    print("Grade B")
else:
    print("Grade C")
```

- Expected Output:

o A reusable function like calculate_grade(marks) with clean logic and docstring.

Prompt Used:

Refactor the following Python code by removing repeated conditional logic.
Create a reusable function calculate_grade(marks) with a proper docstring.
Ensure the grading logic remains unchanged.

Code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")

marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C”)
```

Code:

```
def calculate_grade(marks):
    """
    Calculate the grade based on the marks.

    Parameters:
    marks (float): The marks obtained by the student.

    Returns:
    str: The grade corresponding to the marks.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"
marks_list = [85, 72]
for marks in marks_list:
    grade = calculate_grade(marks)
    print(grade)
```

Output:

```
Grade B
Grade C
```

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

- **Task: Use AI to refactor procedural input–processing logic into functions.**
- Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
```

print("Square:", square)

• Expected Output:

o Modular code using functions like get_input(), calculate_square(), and display_result().

Prompt Used:

Refactor the following procedural Python code into modular functions.
Separate input, processing, and output into functions named get_input(), calculate_square(), and display_result().
Ensure readability improves and functionality remains the same.

Code:
num = int(input("Enter number: "))
square = num * num
print("Square:", square)

Code:

```
def get_input():
    """
    Get a number input from the user.

    Returns:
    int: The number entered by the user.
    """
    return int(input("Enter number: "))
def calculate_square(num):
    """Calculate the square of a number.

    Parameters:
    num (int): The number for which to calculate the square.

    Returns:
    int: The square of the number.
    """
    return num * num
def display_result(square):
```

```
    """Display the result.

    Parameters:
    square (int): The square to display.
    """
    print("Square:", square)
 def main():
    num = get_input()
    square = calculate_square(num)
    display_result(square)
 if __name__ == "__main__":
    main()
```

Output:

```
Enter number: 20
Square: 400
```

Task Description #8 (Refactoring – Optimizing List Processing)

- **Task: Use AI to refactor inefficient list processing logic.**
- Instructions:

o Ask AI to replace manual loops with list comprehensions or built-in functions.

o Ensure output remains identical.

- Sample Legacy Code:

```
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
    squares.append(n * n)
print(squares)
```

- Expected Output:

o Optimized version using list comprehension with improved readability

Prompt Used:

Optimize the following Python code by replacing the loop with a list comprehension.

Ensure the output remains identical and improves readability.

Code:

```
nums = [1, 2, 3, 4, 5]
```

```
squares = []
for n in nums:
    squares.append(n * n)
print(squares)
```